

23AID303 - Quantum Computing and Algorithms

Lab 1 – Basics and Exercises

To work on Jupyter notebook on VSCode

Step 1: Install the Extensions

- Ⓟ Open VS Code
- Ⓟ Go to Extensions on the left Menu and search for Jupyter (by Microsoft) and install
- Ⓟ Search for Python (by Microsoft) and install

Create a folder

Open VS code and go to the folder created

Open new terminal to create an environment (choose cmd on the pull down menu)

Create virtual environment and activate it

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

Upgrade pip if it is needed (in the cmd prompt)

```
python -m pip install --upgrade pip
```

To install IBM Qiskit in the Python Environment

Find if qiskit related packages are present

```
pip list | findstr qiskit
```

(if qiskit is already installed, uninstall and reinstall)

```
pip uninstall -y qiskit
```

also uninstall all packages of qiskit

```
python -m pip install ipykernel
```

```
python -m pip install jupyter ipykernel notebook
```

Then Install Qiskit

```
pip install qiskit
```

```
pip install qiskit qiskit-aer notebook ipykernel matplotlib pylatexenc (all in one command)
```

Change the display name of the virtual env created

```
python -m ipykernel install --user --name lab2026 --display-name "Python (Lab2026)"
```

You should see the (.venv) with your path

Now create a new Jupyter file

To check the Qiskit Version installed

```
import qiskit  
print(qiskit.__version__)
```

To install Aer Simulator module. Aer refers to the high-performance simulator module in Qiskit.

```
pip install qiskit-aer
```

To verify the installation of Aer Simulation module

```
from qiskit_aer import Aer  
print(Aer.backends())
```

To visualize the quantum circuit in mpl format (mpl denotes Matplotlib)

```
pip install pylatexenc  
pip install matplotlib
```

A simple Quantum circuit – Example 1

```
from qiskit import QuantumCircuit  
qc = QuantumCircuit(1)  
qc.h(0)  
print(qc)
```

A simple Quantum circuit – Example 2

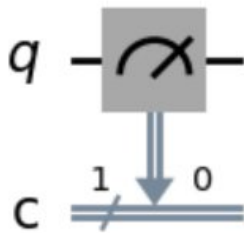
```
from qiskit import QuantumCircuit

qc=QuantumCircuit(1,1) //one quantum bit and 1 classical bit

qc.measure(0,0) // Measures qubit 0 and stores the result in classical bit 0. We cannot
                execute a measure() without a corresponding classical register.

qc.draw(output="mpl")
```

Output



Example 3

```
from qiskit import QuantumCircuit

from qiskit.visualization import plot_histogram

qc = QuantumCircuit(2, 2)

qc.measure([0,1], [0,1]) // qc.measure([0, 1, 2], [0, 1, 2]) for 3 qubit system

print(qc.draw())

display(qc.draw(output="mpl"))

simulator = Aer.get_backend('qasm_simulator')

job = simulator.run(qc, shots=1000)

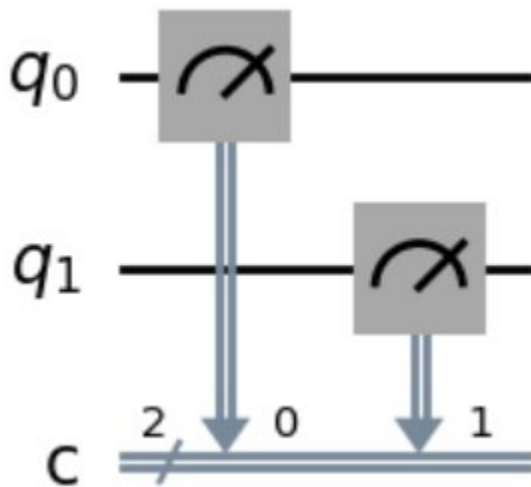
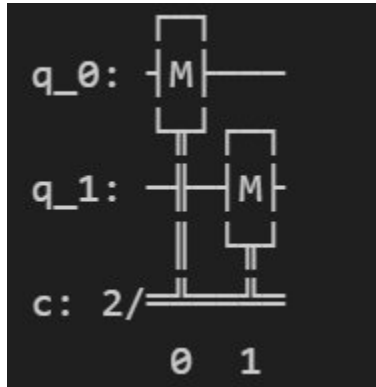
result = job.result()

counts = result.get_counts(qc)

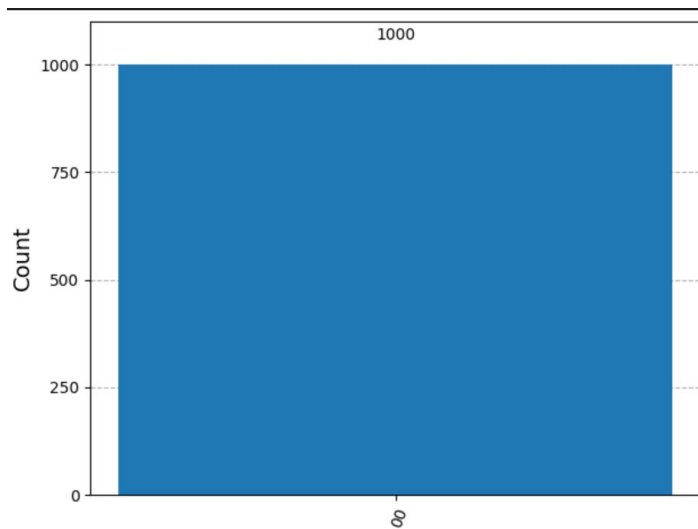
print(counts)

plot_histogram(counts)
```

Output



$\{ '00': 1000 \}$

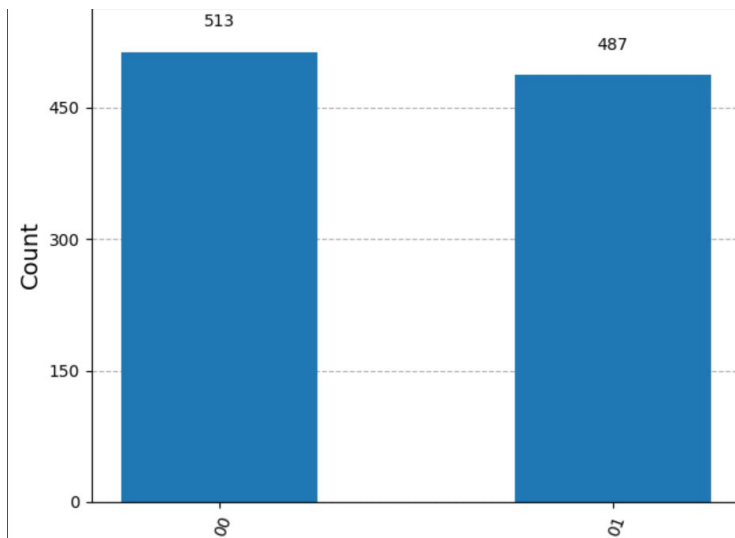


On multiple runs, we see only 00 as there are no quantum gates applied. The qubits start at $|0\rangle \otimes |0\rangle = |00\rangle$ and since no gates are applied, the measurement always remains for 00.

Example 4

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import Aer
from qiskit.providers.basic_provider import BasicProvider
from qiskit.visualization import plot_histogram

qc = QuantumCircuit(2, 2)
qc.h(0) //Superposition applied only to q0. So possible outcomes will be 00 or 10
qc.measure([0, 1], [0, 1])
print(qc.draw())
display(qc.draw(output="mpl"))
simulator = Aer.get_backend('qasm_simulator')
compiled_circuit = transpile(qc, simulator)
job = simulator.run(compiled_circuit, shots=1000)
result = job.result()
counts = result.get_counts(qc)
print(counts)
plot_histogram(counts)
```



[the positions of the bits change..it should be read from right to left]

Now add `qc.h(1)` to the above code. Both the bits will have the outcome of superposition.

Output

`{'11': 256, '00': 264, '01': 241, '10': 239}`

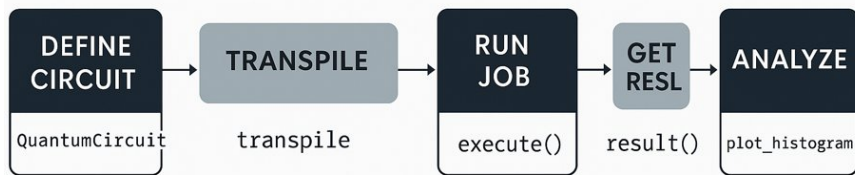
IBM Qiskit Documentation and Details can be found in the link:

<https://www.ibm.com/quantum/qiskit>

The four steps to writing a quantum program using Qiskit patterns are:

1. Map the problem to a quantum-native format.
2. Optimize the circuits and operators.
3. Execute using a quantum primitive function.
4. Analyze the results.

QISKIT PROGRAM LIFECYCLE

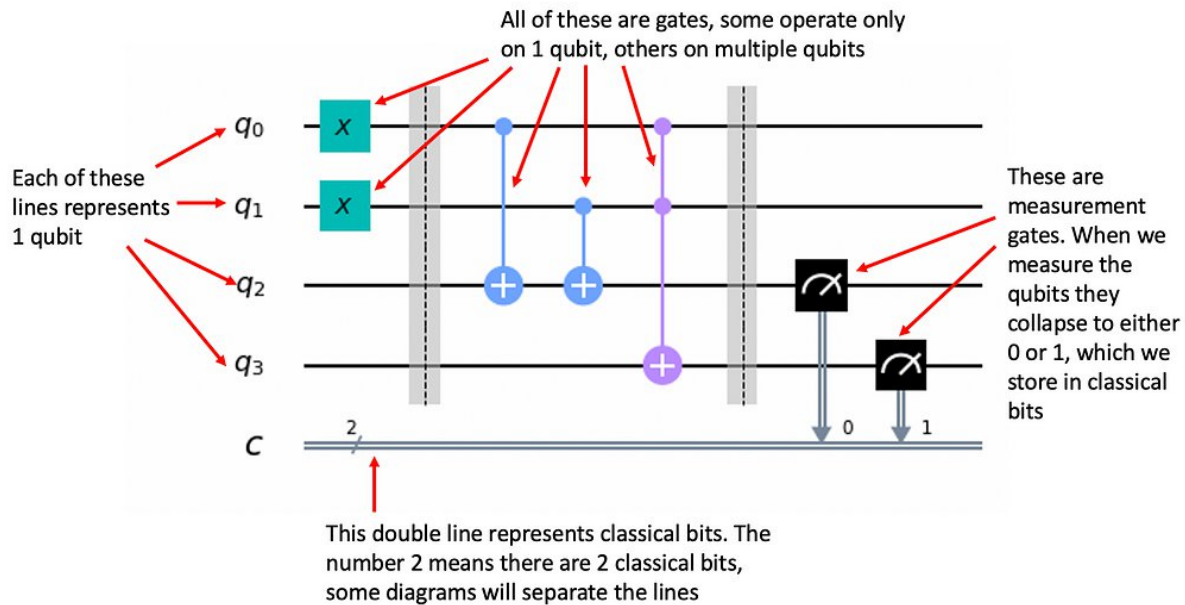


Transpile Vs Compile

- Transpile converts gates to backend-supported gates
- It is an optional step
- It optimizes the circuit with minimum number of gates
- Transpile converts code from one format to another within the same abstraction level (e.g., high-level to hardware-compatible circuit)
- While Compile converts code from high-level language to low-level machine code or bytecode

Feature	Transpile	Compile
Definition	Converts code from one format to another within the same abstraction level (e.g., high-level to hardware-compatible circuit)	Converts code from high-level language to low-level machine code or bytecode
Input	High-level circuit	High-level program or script
Output	Equivalent circuit, optimized for a target backend	Binary or bytecode executable

A sample circuit



Exercises

- Create a 3-qubit quantum circuit. Apply the Hadamard gate to all three qubits and measure them.
 - List all possible outcomes.
 - What is the expected probability of each outcome?
- Given this circuit:


```
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.h(1)
qc.measure([0,1], [0,1])
```

 - What quantum state is prepared before measurement?
 - Write down the full statevector.
 - What are the chances of observing 11?
 - How would the result change if only `qc.h(0)` was used?
- What will happen if you apply the Hadamard gate **twice** to the same qubit?


```
qc = QuantumCircuit(1, 1)
qc.h(0)
qc.h(0)
qc.measure(0, 0)
```

Predict the output using simulation. (Note: Two H gates cancel out and act like the identity.)